**CS 24302 : COMPILER DESIGN LABORATORY**

**Lab Session 3 : Theme :** Application of Pattern Recognition in Problem Solving

**Scheduled : August 20 and 21, 2026**

**Faculty Team** : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranav, Mohd. Shah Fahad, Supreeti Kamilya and Supratim Biswas

**Recap :**

- Given an application domain how to identify its lexical elements and write regular expressions to specify them.
- Write regular expressions for an assorted collection of lexical elements, not only for programming languages, but also for other applications.
- Learnt commands to construct a working scanner for the tokens of an application. Write C codes in the actions part and other designated sections to solve the given problem.
- Experiment with the generated scanner.

**Objectives :** 1. How to write more than one distinct context free grammars to generate a given set of strings of interest.
2. Learn the basic structure of a yacc script and the type of information that is to be supplied in the different placeholders.
3. Get familiar with the manner in which terminals and nonterminals are specified in yacc file. Understand how tokens are communicated between a scanner and its parser.
4. Write an accompanying lex script to recognize and supply the tokens required by the grammar. Use both the scripts to generate a functional parser.
5. Learn a few basic means of communication between yylex() and yyparse(). Examine the contents of the files, “y.tab.h” and “y.output”, that is created by yacc.
6. Experiment with the generated parser and examine its behaviour.

**Lab Assignment 4** : Write a Context Free Grammar to generate a set of strings. Rewrite the grammar as a yacc script, write a lex script for the terminals and generate a scanner and a parser. Study the behaviour of the parser experimentally.

**General Instructions** :
1. Given a "lex-script" (file with extension ".l"), and a "yacc-script" (file with extension ".y"), a functional scanner and parser are generated using the following commands. Recall that lex constructs the file "lex.yy.c" which defines the function yylex(). The s/w yacc generates a C program, "y.tab.c" which defines the function yyparse(), besides other definitions.

| Generation of scanner and parser | Test the generated Parser |
|---|---|
| $ lex lex-script<br>$ yacc yacc-script<br>$ gcc lex.yy.c y.tab.c -o parser | $ ./ parser < input.txt > output.txt<br>where "input.txt" is file containing input and the results produced by the parser is saved in the file, "output.txt" |

Note that yacc generates a special bottom-up parser which we shall be explained during the theory lectures.
"

**P1.** You are given 2 scripts, decl1.l" and "decl1.y". Read the contents of both the files. Read the document, "yacc-script-structure,pdf" to get a feel of a yacc script.
(a) Generate the scanner and parser and test your parser on the input file, "inp1.txt".
(b) Explain each element of the output and write each method used for communication between the scanner and the parser.
(c) Construct two different inputs, each having single declaration, for which the parser outputs syntax error at different input symbols. Explain the error message issued by the parser.

**Resources Given** : "decl1.l", "decl1.y", "inp1.txt", "yacc-script-structure,pdf"

**P2**. The grammar of P1 can handle a single declaration only. You have to modify lex and yacc scripts so that the parser generated by you produces output for the input given below. Note that your parser should be able to handle arbitrary number of declaration statements with the types {int, float, char}.

| | |
|---|---|
| int i, j;<br>char comma, _semicolon;<br>float pi, g, comma;<br>int i, j; | Data type is int<br>First Identifier is i<br>Next Identifier is j<br>1 Declaration statements parsed successfully<br>Data type is char<br>First Identifier is comma<br>Next Identifier is _semicolon<br>2 Declaration statements parsed successfully<br>Data type is float<br>First Identifier is pi<br>Next Identifier is g<br>Next Identifier is comma<br>3 Declaration statements parsed successfully<br>Data type is int<br>First Identifier is i<br>Next Identifier is j<br>4 Declaration statements parsed successfully |

**Resources Given** : "inp21.txt", "out21.txt"

**P3.** The CFG of P2 requires a recursive rule to generate the strings of interest. Change the recursive rule(s) used in P2 from left recursive to right recursive or conversely as appropriate. Generate a new parser and observe the similarity and / or differences in behaviour as compared to the parser of P2.

**End of the Experiment 4**